Rapport TP2 LOG725 – Refactorisation ECS

par

Alice INVERNIZZI

RAPPORT DE PROJET
PRÉSENTÉ À L'ÉCOLE DE TECHNOLOGIE SUPÉRIEURE DANS LE CADRE DU COURS
LOG725

MONTREAL, LE 4 DECEMBRE 2025
SESSION AUTOMNE

ECOLE DE TECHNOLOGIE SUPÉRIEURE
UNIVERSITÉ DU QUÉBEC

 Alice Invernizzi, 2025

Table des matières

Rapport TP2 LOG725 – Refactorisation ECS 1
I. Mise en contexte 3
II. Architecture avant refactorisation 3
1) Diagramme de classes 3
2) Problèmes identifiés 4
III. Architecture après refactorisation 4
1) Vue d'ensemble 4
2) Architecture ECS 5
3) Patron de conception Command 7
4) Patrons Observer et State 8
IV. Comparaison avant/après 11
V. Conclusion 11

I. Mise en contexte

Ce rapport présente la refactorisation complète d'un jeu Snake en C#, en Windows Forms, dans le but d'implémenter une architecture ECS (Entity-Component-System), avec plusieurs patrons de conception afin d'améliorer la maintenabilité et la flexibilité du code.

Le projet original utilise une approche orientée objet classique. La refactorisation effectuée lors de ce TP permet de découpler la logique de jeu et de faciliter l'ajout de nouvelles fonctionnalités potentielles dans le futur, tout en conservant le rendu final du jeu.

Le jeu original provient du repository GitHub accessible au lien suivant :

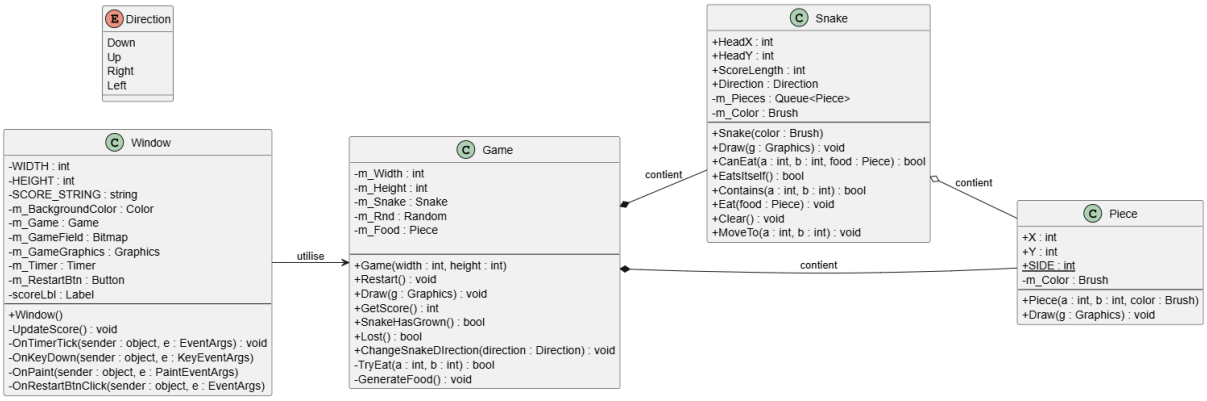
https://github.com/victoriazakharo/Snake

Le code original a été conservé, notamment dans le dossier Snake. **Important** : **La refactorisation a été effectuée dans le dossier Snake_v2.**

Ce choix a été fait pour la simplicité du jeu, qui possède une logique suffisamment simple pour être refactorisé en profondeur, ainsi que pour sa pertinence (il a été assez simple d'imaginer l'architecture ECS à implémenter sur les mécaniques du jeu) et son évolutivité, permettant possiblement d'ajouter facilement des nouvelles fonctionnalités lors d'une implémentation future.

II. Architecture avant refactorisation

1) Diagramme de classes



2) Problèmes identifiés

Avec l'architecture originale, le couplage entre les classes est fort. La classe Game vient gérer directement la logique métier, le rendu graphique, la génération de nourriture et la détection de collisions, tandis que la classe Snake contient les données (positions), le code graphique et la logique (mouvement, collision). Dans l'idée, avec une telle organisation, il est difficile de tester unitairement les différentes parties du code, même si dans notre cas le jeu reste relativement simple et court.

On observe également un problème de responsabilités multiples : Snake ne respecte pas le principe de responsabilité unique puisqu'elle gère à la fois données, logique, rendu. Game possède également plusieurs responsabilités différentes, comme mentionné précédemment.

Les segments du Snake ne sont pas des entités indépendantes et sont enfermés dans l'objet Snake, ce qui limite la modularité et rend difficile l'implémentation de comportements réutilisables, ou même l'ajout de nouveaux types d'entités.

De plus, l'extensibilité est limitée. Ajouter de nouveaux types d'entités nécessiterait de modifier plusieurs classes. Il n'est pas possible de réutiliser facilement des comportements entre différentes entités.

Et finalement, la gestion des entrées clavier est directement codée dans Window.OnKeyDown() avec un switch/case, ce qui rend difficile l'ajout de nouvelles commandes et la personnalisation des touches notamment. La gestion des entrées pourrait être modifiée.

III. Architecture après refactorisation

1) Vue d'ensemble

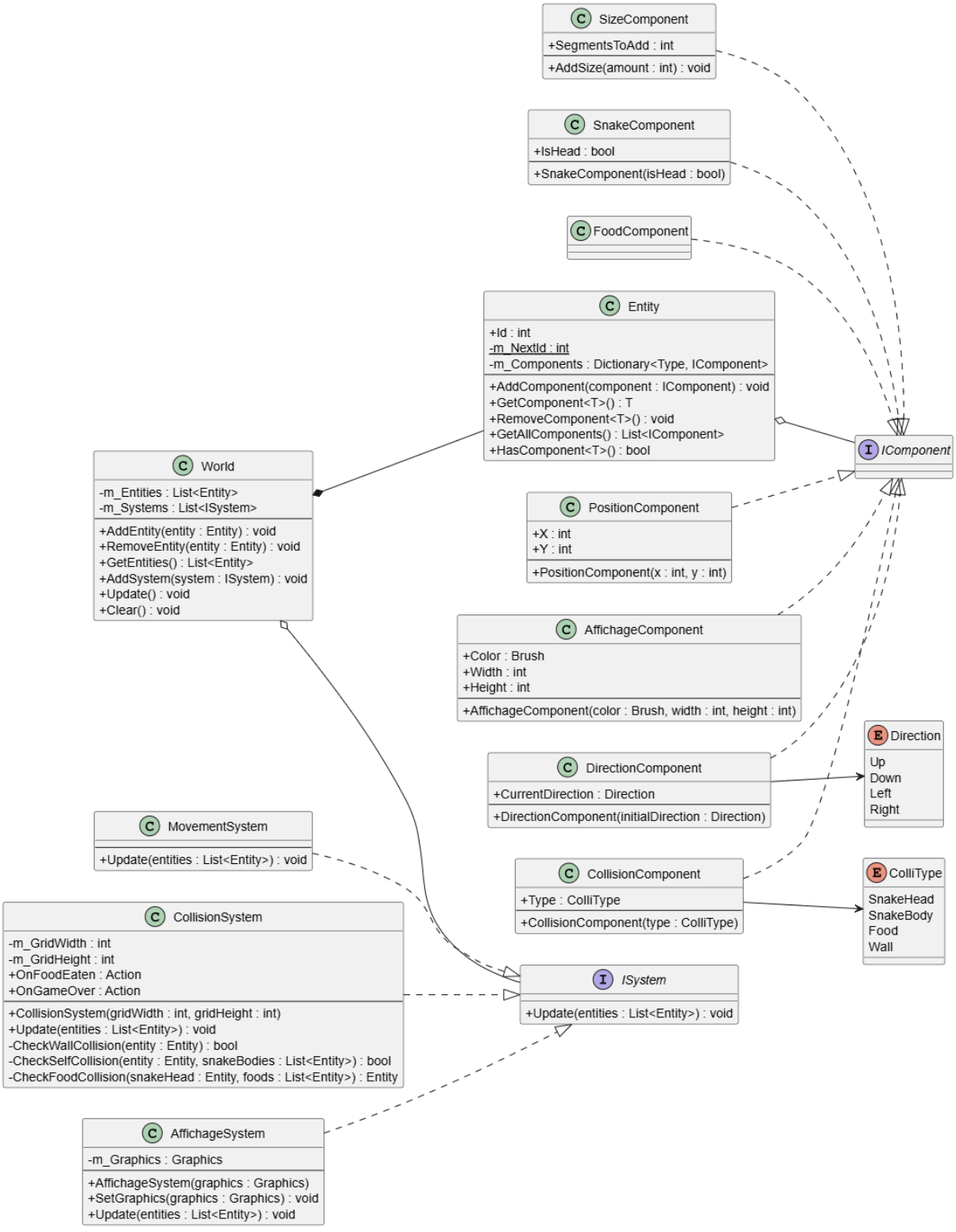
L'architecture refactorisée repose sur 3 éléments principaux qui permettent une séparation claire des responsabilités et une meilleure organisation du code :

- L'architecture **ECS** (Entity-Component-System) pour la gestion des entités du jeu et de leur logique
- Le **patron Command** pour le découplage des entrées utilisateur et de leurs actions
- Les **patrons Observer et State** pour la communication entre modules et la gestion des états du jeu

Cette nouvelle organisation permet d'isoler les données (composants), la logique (systèmes) et le comportement (états), facilitant ainsi la maintenance, les tests et l'évolution future du code.

Par la suite, nous allons présenter 3 diagrammes de classes de la refactorisation, par souci de clarté.

2) Architecture ECS



L'architecture ECS repose sur trois concepts fondamentaux qui transforment radicalement l'organisation du code.

Entity (Entité) : Conteneur léger identifié par un ID unique. Une entité ne contient aucune logique, seulement une collection de composants qui définissent ses caractéristiques. Dans notre implémentation, chaque segment du serpent est une entité distincte, tout comme la nourriture. Cette approche permet une grande flexibilité : ajouter un obstacle ou un power-up ne nécessiterait que la création d'une nouvelle entité avec les composants appropriés.

Component (Composant) : Structure de données pure qui ne contient aucune logique. Les composants définissent les caractéristiques d'une entité :

- **PositionComponent** : coordonnées X,Y sur la grille
- **DirectionComponent** : direction actuelle du mouvement
- **AffichageComponent** : couleur et dimensions pour le rendu
- **SnakeComponent** : marqueur identifiant les segments du serpent (avec un booléen IsHead pour différencier la tête)
- **CollisionComponent** : type d'entité pour la gestion des collisions
- **SizeComponent** : gestion de la croissance du serpent
- **FoodComponent** : marqueur identifiant la nourriture

System (Système) : Contient toute la logique métier et opère sur les entités possédant certains composants :

- **MovementSystem** : gère le déplacement du serpent en modifiant les PositionComponent selon la DirectionComponent
- **CollisionSystem** : détecte les collisions (murs, auto-collision, nourriture) et déclenche les événements appropriés
- **AffichageSystem** : effectue le rendu visuel de toutes les entités possédant PositionComponent et AffichageComponent

Le **World** coordonne l'ensemble. Il stocke toutes les entités et il gère l'exécution séquentielle des systèmes à chaque frame.

L'architecture ECS impacte toutes les mécaniques fondamentales du jeu :

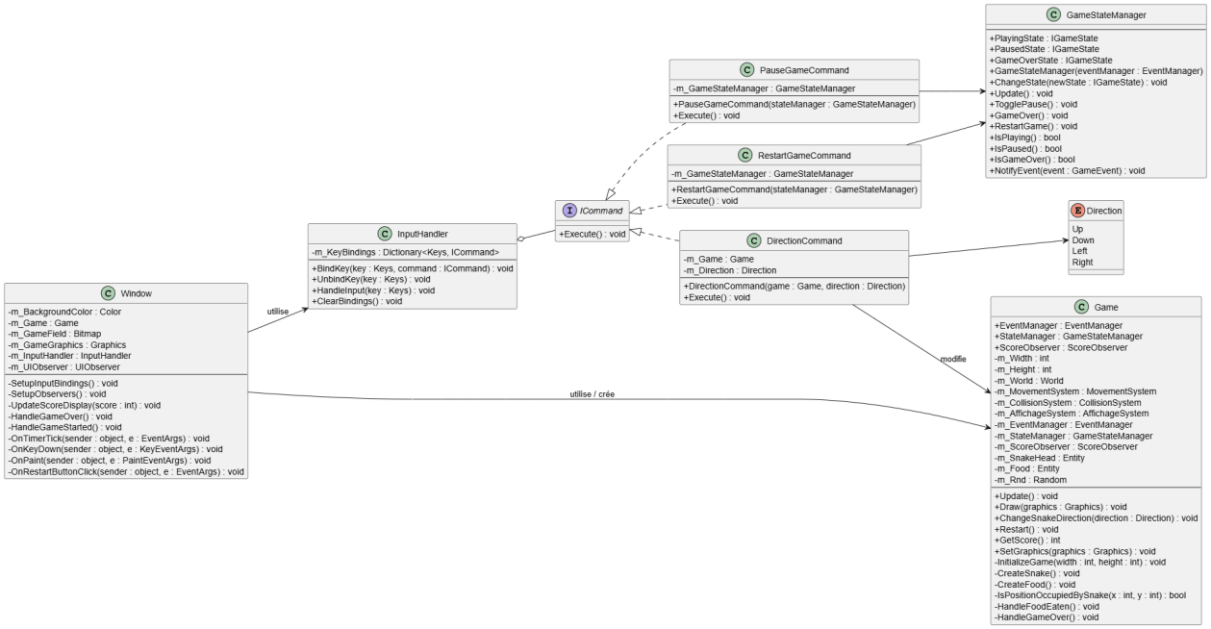
Déplacement du serpent : le MovementSystem parcourt les entités possédant SnakeComponent, sauvegarde les positions actuelles, déplace la tête selon la direction, puis propage le mouvement dans le corps en décalant chaque segment à la position du segment précédent. Lors d'une croissance (après avoir mangé), un nouveau segment est créé et ajouté dynamiquement.

Détection des collisions : le CollisionSystem vérifie trois types de collisions : la collision avec les murs (position hors limites), l'auto-collision (tête qui touche un segment du corps) et la collision avec la nourriture (qui déclenche la croissance).

Rendu visuel : le AffichageSystem itère sur toutes les entités ayant les composants nécessaires et dessine un rectangle à leur position avec leur couleur respective.

Cette séparation permet d'ajouter facilement de nouvelles mécaniques sans modifier le code existant, simplement en créant un nouveau système.

3) Patron de conception Command



Le patron Command encapsule les actions du joueur dans des objets de commande, permettant un découplage complet entre les entrées et leurs effets.

Problème résolu : Dans la version originale, la méthode Window.OnKeyDown() contenait un switch/case gérant directement les touches. Ajouter une nouvelle action ou personnaliser les contrôles nécessitait de modifier cette méthode, violant le principe Open/Closed.

Implémentation : L'interface ICommand définit une méthode Execute(). Les commandes concrètes implémentent cette interface :

- **DirectionCommand** : change la direction du serpent
- **PauseGameCommand** : bascule entre pause et jeu
- **RestartGameCommand** : redémarre une partie

L'InputHandler maintient un dictionnaire <Keys, ICommand> mappant les touches aux commandes, et la méthode BindKey() permet d'associer dynamiquement une touche à une commande.

Utilisation : Dans Window.SetupInputBindings(), les touches sont configurées.

```
67          // Lier les touches aux commandes
68          m_InputHandler.BindKey(Keys.Up, upCommand);
69          m_InputHandler.BindKey(Keys.Down, downCommand);
70          m_InputHandler.BindKey(Keys.Left, leftCommand);
71          m_InputHandler.BindKey(Keys.Right, rightCommand);
72          m_InputHandler.BindKey(Keys.W, upCommand);
73          m_InputHandler.BindKey(Keys.S, downCommand);
74          m_InputHandler.BindKey(Keys.A, leftCommand);
75          m_InputHandler.BindKey(Keys.D, rightCommand);
76          m_InputHandler.BindKey(Keys.P, pauseCommand);
77          m_InputHandler.BindKey(Keys.Space, pauseCommand);
78          m_InputHandler.BindKey(Keys.R, restartCommand);
```

Lors d'un appui sur une touche, Window.OnKeyDown() délègue simplement à InputHandler.HandleInput() qui exécute la commande associée.

Les **mécaniques touchées par Command** sont **:**

- **Changement de direction** : les flèches et WASD sont mappés aux commandes de direction
- **Pause/Reprise** : les touches P et Espace basculent l'état du jeu
- **Redémarrage** : la touche R relance une nouvelle partie

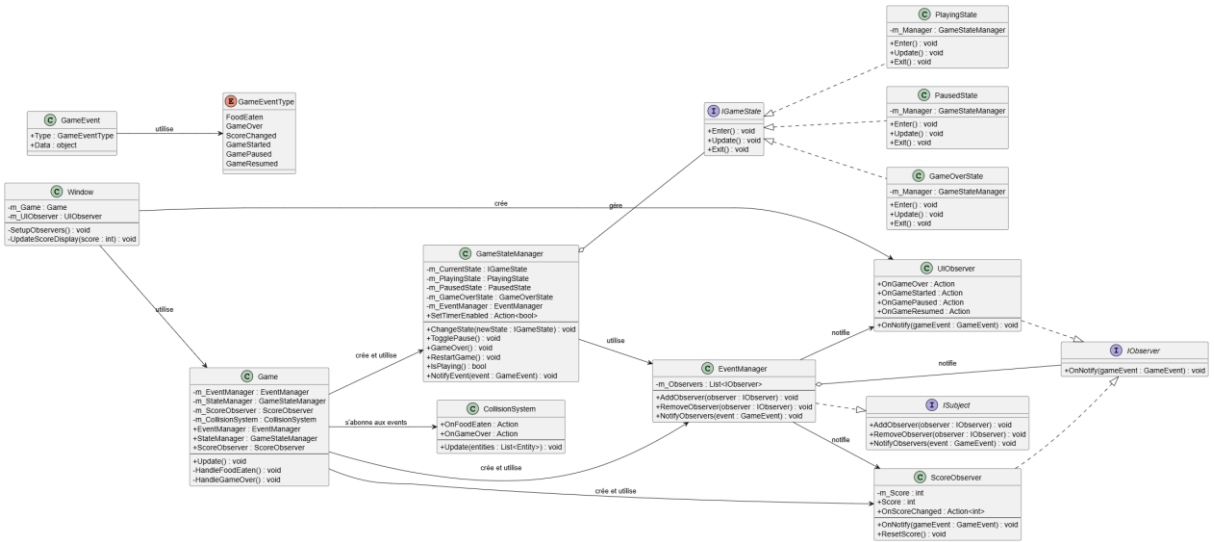
De plus, il y a des **avantages** à l'utilisation du patron Command, tels que :

- **L'extensibilité** : ajouter une nouvelle commande ne nécessite qu'une nouvelle classe implémentant ICommand, sans toucher au code existant.
- La **flexibilité** : le remapping des touches est trivial. Un menu de configuration pourrait facilement permettre au joueur de personnaliser ses contrôles, sur une version peut-être plus évoluée avec des menus
- La **testabilité** : chaque commande peut être testée indépendamment en appelant simplement Execute()

Cependant, dans notre cas, il y a quelques **inconvénients**, notamment :

- La complexité accrue : pour un jeu simple, la création d'une classe par commande peut sembler disproportionnée comparée à un simple switch/case. Ce travail accompli serait, en conditions réelles, utile pour un cas d'amélioration prévue du jeu
- Le nombre de fichiers : augmentation du nombre de classes et de fichiers dans le projet.

4) Patrons Observer et State



a) Observer

Le patron Observer établit une relation de dépendance un-à-plusieurs où plusieurs "observateurs" sont automatiquement notifiés des changements d'état d'un "sujet".

Problème résolu : Dans la version originale, `Window` devait constamment interroger `Game` pour savoir si le score avait changé ou si la partie était terminée (polling). Cela créait un couplage fort et une communication inefficace.

Implémentation :

- L'interface ISubject définit les méthodes AddObserver(), RemoveObserver() et NotifyObservers()
- L'EventManager implémente ISubject et maintient une liste d'observateurs
- L'interface IObserver définit OnNotify(GameEvent) que tous les observateurs doivent implémenter
- La classe GameEvent encapsule un type d'événement (GameEventType) et des données optionnelles

Observateurs concrets :

- **ScoreObserver** : écoute FoodEaten pour incrémenter le score et GameStarted pour le réinitialiser. Il expose un événement C# OnScoreChanged permettant à l'UI de réagir immédiatement.
- **UIObserver** : réaction aux événements GameOver, GameStarted, GamePaused, GameResumed en exposant des actions C# que Window peut écouter.

Le **flux d'événements** est de la forme suivante. Le CollisionSystem détecte qu'une nourriture est mangée, donc ça déclenche OnFoodEaten. Game.HandleFoodEaten() notifie l'EventManager avec un événement FoodEaten, qui vient appeler OnNotify() sur tous les observateurs enregistrés. Ensuite, le ScoreObserver incrémente le score et déclenche son propre événement, puis Window reçoit la notification et met à jour l'affichage

Les **mécaniques touchées par le patron Observer** sont :

- **Mise à jour du score** : communication immédiate et découplée entre la logique de jeu et l'affichage
- **Notifications UI** : activation/désactivation du bouton Restart selon l'état du jeu
- **Synchronisation** : toutes les composantes restent synchronisées sans dépendances directes

Les **avantages** remarqués à l'utilisation du patron Observer sont notamment le **découplage fort,** puisque désormais Game n'a pas connaissance de Windows, de l'UI, et il notifie seulement les événements. De plus, c'est la **communication**, avec le fait que les changements sont notifiés

instantanément ce qui élimine le besoin de polling. Il y a également l'**extensibilité**, qui fait qu'ajouter un nouvel observateur est facile, sans besoin de modifier le code existant, ou encore le principe de **responsabilité unique** qui est vérifié par le fait que chaque observateur gère un aspect spécifique. Enfin, il y a le fait qu'un événement peut déclencher plusieurs actions dans différentes parties du système.

Les **inconvénients** seraient plutôt la **complexité de gestion**, car il faut correctement gérer les abonnements/désabonnements pour éviter les fuites. Aussi, on peut avoir des **difficultés de débogage** car le flux d'exécution est moins évident, puisque les notifications traversent plusieurs couches. D'ailleurs, l'ordre de notification peut créer des bugs si plusieurs observateurs modifient un état partagé. Et finalement, il y a également le fait que pour chaque événement, tous les observateurs vont être notifiés même s'ils n'y réagissent pas, ce qui n'est pas non plus grave.

b) State

Le patron State permet à un objet de modifier son comportement lorsque son état interne change, donnant l'impression que l'objet change de classe.

Problème résolu : La gestion des états du jeu (en cours, en pause, game over) était dispersée avec des flags booléens. Ajouter de nouveaux états ou gérer les transitions devenait complexe et pouvait être source d'erreurs.

Implémentation :

- L'interface IGameState définit Enter(), Update() et Exit()
- 3 états concrets implémentent cette interface :
    - **PlayingState** : jeu actif, timer en marche, tous les inputs sont acceptés
    - **PausedState** : jeu figé, timer arrêté, seuls certains inputs (reprise, restart) sont acceptés
    - **GameOverState** : partie terminée, attente du redémarrage
- Le GameStateManager maintient une référence à l'état courant et gère les transitions

Chaque état définit son propre comportement dans Enter() (activation/désactivation du timer, notification des événements) et gère différemment les méthodes de transition.

Les **mécaniques touchées par le patron State** sont :

- **Pause/Reprise** : transition fluide entre les états avec les comportements appropriés
- **Fin de partie** : passage automatique à l'état GameOver lors d'une collision
- **Redémarrage** : réinitialisation complète et retour à l'état Playing
- **Contrôle du timer** : chaque état gère son propre comportement envers le timer

Les **avantages** trouvés à l'utilisation de State sont notamment le fait d'avoir des **états explicites**, puisque chaque état est maintenant une classe distincte qui a un comportement défini. De plus, les **changements d'état sont explicites** via ChangeState() ce qui facilite le débogage. Il n'y a plus forcément besoin des conditions imbriquées, et ajouter un nouvel état ne nécessite qu'une nouvelle classe ce qui augmente la **maintenabilité.** Par ailleurs, on peut ajouter des états sans modifier ceux qui existent déjà, et chaque état définit comment il réagit aux mêmes appels.

En **inconvénients,** il y aurait l'**augmentation du nombre de classes** car il y a une classe par état, ce qui est un peu excessif pour un jeu simple avec peu d'états, comme cette version initiale de Snake. Aussi, la mise en place du pattern nécessite plus de code qu'une simple variable d'état. Pour un jeu aussi simple, la gestion avec des booléens aurait pu suffire, c'est un peu un cas **d'over-engineering**. Et finalement, il faut bien concevoir quelles transitions sont possibles pour éviter les états incohérents, ce qui est assez **strict**.

IV. Comparaison avant/après

Finalement, voici un tableau récapitulatif des changements apportés par la refactorisation.

	Avant refactorisation	**Après refactorisation**
Nombre de classes	5 classes principales	Beaucoup de classes
Couplage	Fort	Faible
Testabilité	Difficile	Facile (isolation)
Extensibilité possible	Limitée	Haute
Responsabilités	Mélangées	Séparées
Gestion des inputs	Switch/case rigide	Pattern command
Communication	Couplage direct	Pattern Observer
Etats du jeu	Beaucoup de flags booléens	Pattern State

V. Conclusion

Avec la refactorisation effectuée, nous avons pu transformer l'architecture du projet en séparant clairement les responsabilités. L'ECS vient isoler les données et la logique, tandis que le pattern Command vient structurer la gestion des inputs et actions. Le pattern Observer gère la communication, et State les transitions d'état.

Pour notre projet, dans une vision d'apprentissage c'était très intéressant. Cependant, dans une optique réelle, apporter une telle modification d'architecture sur un petit projet de ce genre ne serait peut-être pas très pertinente, car elle apporte beaucoup de complexité en nombre de fichiers. Mais tout de même, on constate beaucoup d'avantage en termes de maintenabilité, testabilité, extensibilité, ce qui explique que ce système soit bien utilisé et apprécié de nos jours. Il est également maintenant possible d'ajouter facilement de nouvelles fonctionnalités. Il est donc très utile de coder de cette manière dès le début du projet.